Data Science — Elective Assignment 2026

Neuro-Symbolic Reinforcement Learning & Planning

Subject	Data Science — Neuro-Symbolic AI
Topic No.	10
Submit to	latesh.gagan@gmail.com
Due Date	30th March 2026
Submitted by	Roshan Kumar— Elective Roll No. 55

1. The Transparency Problem in Modern AI

The deep learning industry is gaining a reputation. If there is enough labelled data and compute a well-tuned network will find structure that no human analyst would manually find. Basically it's a fraud detection system a protein folding system a classification of images. There is no question that these models work. Is there a reason why these things work? Why do they work? The problem is that nobody can say exactly why they work and that distinction starts to matter the moment they're put there. An applicant for a credit card who rejects a loan application must give a reason. If the tumour is detected by a doctor the doctor will point it out to the patient. If a section of bridge fails the structural engineer must refer to the measurement in order to prove it. It is not arbitrary but the way accountability works in the professional practice. But it does nothing like Deep RL. In this case the model outputs an answer and does not provide any traceable logic.

No Transparency	Data Hunger	Brittle to Change
When a pure deep RL is pushed into real deployments [2][6]three failure modes continue to appear in practice:No TransparencyData HungerBrittle to ChangeA deep RL controller for railway signalling generates millions of nested weight interactions The engineers can not diagnose it in time to prevent an incident when it fails. Regulators have to ensure auditability before deployment.	It takes months of simulation runs to train a warehouse sorting robot. Moving to another warehouse different rack heights different rotation radii restarts the process from scratch. The algorithm tuned for volatility patterns before 2020 collapsed when the market structure changed. The model didn't have a mechanism to detect that the environment had changed it just continued to execute a policy that no longer fit. [12]	A trading algorithm tuned on pre-2020 volatility patterns collapsed when the market structure shifted. The model had no mechanism to detect that the environment had changed — it just kept executing a policy that no longer fit. [12]

Each of these failures has the same root cause: the model learns correlations from data but does not hold any explicit representation of how the world works. When the world changes, there is nothing to update — only a retraining job.

2. Symbolic Deep Reinforcement Learning

The obvious solution is to add a rule layer something that describes the world in terms a human could read and verify. In response to the rigidity of pure rule systems there is a clear reaction: learning. Both Neuro-Symbolic RL and the combination is not just additive. Each halves constrain and guide each other in ways neither can alone achieve. New obstructions, slow conveyors, pickers calling sick: the controller adapts without changing the plan. If the controller actually

meets the symbolic conditions set by the planner it should be checked. A meta controller will revise the plan instead of waiting for the entire operation to collapse if an operation takes too long, violates a constraint or is heading for failure.

Perception & Pattern Recognition	NS-RL	Logic & Hard Constraints
Sensor data images, unstructured signals the neural layer finds patterns too complex for any hand-written rule	NS-RLPerception feeds proposals upward the symbolic layer filters, verifies and plans learning fills the gaps	Given an objective say a package from intake to dispatch in a logistics centre the planner maps the route through sub-goals using known domain rules: capacity limits priority queues exclusion zones etc

These rules can't be learned by domain knowledge. The agent of Deep RL handles the physical execution of the agent. Through trial and experience the planner learns how to carry out each set of sub-goals under real-world variations. New obstructions, slow conveyors, pickers calling sick: the controller adapts without changing the plan. If the controller actually meets the symbolic conditions set by the planner it should be checked. A meta controller will revise the plan instead of waiting for the entire operation to collapse if an operation takes too long, violates a constraint or is heading for failure. The diagram below illustrates where each component :

1	**Planner**	Works completely in symbols. Given an objective say a package from intake to dispatch in a logistics centre the planner maps the route through sub-goals using known domain rules: capacity limits priority queues exclusion zones etc. It cannot learn these rules they come from domain knowledge.
2	**Controller**	The Deep RL agent handles physical execution. For each set of sub-goals the planner learns through trial and experience how to carry them out under real-world variation. New obstacles, slow conveyors, pickers calling sick: the controller adapts without changing the planner.
3	**Meta Controller**	If the controller is actually fulfilling the symbolic conditions set by the planner it should be checked. If an operation takes too long, violates a constraint or is heading for failure the Meta Controller revises the plan rather than waiting for the entire operation to collapse.

3. Three Key Ideas in NS-RL

3.1 Interactive World Modelling

Three Key Ideas in Interactive World Modelling in NS-RL3.1Large scale language models are hallucinate because they generate output from a distributed distribution and that distribution does not know when it has wandered outside the boundary of facts.

But the model doesn't know that he's making it up. It works differently in the energy function. It specifies a specific set of conditions that the output must meet. If there's even one condition broken then the generation ceases. Think less about a prompt telling the model what to write and more like a circuit breaker connected directly to the generation process.

An example of air traffic management:

End-to-End Example — Air Traffic Collision Avoidance (NS-RL system)	
State:	Flight AA302 is 8nm away, descending, closure rate above threshold

Symbolic Policy: IF Closure_Rate > 300 knots AND Altitude_Delta < 500ft
THEN Action = CLIMB

Translation: "The agent issued a climb instruction because the vertical separation fell below the minimum safe threshold while closure rate exceeded limits."

The key difference from prompt engineering is where the rules live. Prompts are guidelines on the surface. Energy function constraints are built into the probability distribution itself. The model cannot output a rule-violating token — it is filtered before it can form.

3.2 Deep Symbolic Policy Generation

This section is about something different — not constraining a model that already exists, but building a control policy that is itself a readable mathematical expression. Instead of a neural network that maps states to actions through millions of unreadable weights, the system searches for a formula.

Component	Description
Neural Generator	An autoregressive RNN that treats mathematical operators (+, *, sin, log) as a vocabulary — it literally writes equations symbol by symbol, building a syntax tree. [8]
Symbolic Policy	The output is something like: Throttle = 0.7 * sin(Speed) - 0.3 * Curvature. This is the actual control law — readable, auditable, deployable. [7][8]
Evaluator	The candidate formula is tested in a physics simulator. A drone trajectory policy, for instance, is checked against real wind resistance and rotor dynamics.
Update Mechanism	Risk-Seeking Policy Gradients — instead of optimising for average performance across all test runs, the system chases the best run it ever achieved. This pushes the formula toward peak performance rather than mediocre reliability. [7][8]

3.3 Bilevel Hierarchical Planning [11][14][15]

Most RL systems start from zero on every new task. In the case of bilevel planning argues that this is wasteful and unnecessary especially when the domain already has structured knowledge. In fact a robot that has never worked in a kitchen can use the known layout of a kitchen (stove here, sink there, fridge across) as a scaffold to learn individual tasks.

It works in two layers:

- The upper level holds symbolic transition models and then rules about how the world changes with certain actions. Most of these are built from domain expertise or previous datasets and not learned from scratch.
- The lower level is where the neural RL is performed. Each symbolic transition learns how to physically execute each symbolic transition in a real messy environment handling noise timing and variation.
- In case of a new task, the system maps it to the existing symbolic scaffold and fills only what is genuinely new. The transfer of information is efficient because the agent is not learning the structure of the world but rather the details of how to act in it.

4. Why This Matters — Real World Impact

NS-RL is at this point not a theoretical proposal. A number of real systems have been built around these ideas and the results are measurable:

- AlphaZero and MuZero (DeepMind) neural evaluation of board positions combined with symbolic game rules to achieve superhuman performance in chess It is not learnt that the symbolic rules are hard constraints within which the neural component operates. [14]
- AlphaGeometry (2024) — solved 25 out of 30 problems from the International Mathematical Olympiad. The setup paired a neural model that suggests geometric constructions with a symbolic geometry engine that verifies whether each suggestion is logically valid. Neither component alone could crack more than a handful of problems.
- Industrial Robot Safety: symbolic constraints can give provable guarantees that a robotic arm will never move into a zone where a human operator is present. A reinforcing neural controller can't give this guarantee it can only give probability.
- Medical Diagnosis Support — Rule layers for Medical Diagnosis Support can enforce clinical guidelines (e.g. the neural layer handles the pattern recognition across thousands of patient variables).
- Modern AI Agents — when Claude, GPT-4 or Gemini are given a calculator, a code interpreter or a database query tool, that is neuro-symbolic integration in practice. The LLM handles language and reasoning; the tool handles formal execution.

5. Conclusion

ConclusionIt's not that it solves everything it's that it solves specific things which pure neural and pure symbolic approaches cannot solve on their own. Neural networks are powerful at perception and generalisation but they cannot explain themselves or guarantee behaviour. Symbols are transparent and reliable but cannot handle raw sensory data or adapt to environments they were not specifically programmed for.

Each of the three approaches covered here interactive world modelling deep symbolic policy generation and bi-level hierarchical planning are attacking one of these gaps from a different perspective. The world model adds verifiable constraints to generative systems. Symbolic policy generation makes the control law itself understandable. This allows for structured knowledge transfer rather than relearning.

What makes this worth following is that the results are not marginal. AlphaGeometry beating experienced mathematicians on Olympiad problems is not an incremental improvement — it is a qualitative jump that neither approach alone could have produced. If that kind of result extends to logistics, medical systems and infrastructure management, the hybrid architecture will not stay academic for long.

References

[1] Minsky, M. (1991). Logical vs. analogical or symbolic vs. connectionist or neat vs. scruffy. AI Magazine, 12(2), 34–51.

[2] Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

[3] Garcez, A. d. A., & Lamb, L. C. (2023). Neurosymbolic AI: The 3rd wave. Artificial Intelligence Review, 56, 12387–12406.

[4] Luo, Y. et al. (2024). End-to-end neuro-symbolic reinforcement learning with textual explanations (INSIGHT). arXiv:2403.12451.

[5] Wan, Z. et al. (2024). Towards cognitive AI systems: A survey and prospective on neurosymbolic AI. arXiv:2401.01040.

[6] Acharya, A. et al. (2023). Neurosymbolic reinforcement learning and planning: A survey. arXiv:2309.01038.

[7] Verma, A. et al. (2018). Programmatically interpretable reinforcement learning (PIRL). arXiv:1804.02477.

[8] Landajuela, M. et al. (2021). Discovering symbolic policies with deep reinforcement learning. ICML 2021.

[9] Lyu, D., Yang, F. et al. (2019). SDRL: Interpretable and data-efficient deep RL leveraging symbolic planning. arXiv:1811.00090.

[10] Yang, F., Lyu, D. et al. (2018). PEORL: Integrating symbolic planning and hierarchical RL for robust decision-making. IJCAI 2018.

[11] Chitnis, R. et al. (2022). Learning neuro-symbolic relational transition models for bilevel planning. IROS 2022. arXiv:2105.14074.

[12] Balloch, J. C. et al. (2023). Neuro-symbolic world models for adapting to open world novelty. arXiv:2301.06294.

[13] Agravante, D. J. et al. (2023). Learning neuro-symbolic world models with conversational proprioception. ACL 2023.

[14] Silver, D. et al. (2018). A general RL algorithm that masters chess, shogi and Go through self-play (AlphaZero). Science, 362(6419).

[15] Schrittwieser, J. et al. (2020). Mastering Atari, Go, chess and shogi by planning with a learned model (MuZero). Nature, 588.

[16] Delfosse, Q. et al. (2023). Interpretable and explainable logical policies via neurally guided symbolic abstraction (NUDGE). arXiv:2306.01439.